Data Quality Assessment for the Formula 1 Reconciled Database

Giuseppe Rudi

May 8, 2026

Contents

Introduction

This document describes the Data Quality Assessment phase applied to the Formula 1 reconciled database. The goal of this phase is not to clean the data directly, but to evaluate the current quality level of the data stored in PostgreSQL before applying cleaning rules and before loading the final Data Warehouse.

Since the Data Warehouse and the final analytical dashboards depend on the quality of the reconciled level, a systematic assessment is required before continuing with the cleaning and ETL phases.

In this project, Data Quality Assessment is implemented as a deterministic and reproducible Python process that reads the reconciled tables from PostgreSQL, applies a set of quality checks, and produces scorecards and row-level issue files.

Considerations

During the first DQA iteration, some issues were not caused by the original data source but by the re-engineering process. In particular, the DQA helped detect problems related to key uniqueness, referential consistency, data quality rules. These issues were corrected at pipeline level by modifying the re-engineering logic and rebuilding the reconciled database before running the final DQA.

Role of Data Quality Assessment

The Data Quality Assessment phase acts as an audit checkpoint between the reconciled database and the cleaned analytical data. Its purpose is to detect data problems before they propagate to the Data Warehouse, the star schema, and the final visual analytics layer.

The project pipeline can be summarized as follows:

Raw and external sources
↓
Extraction and re-engineering
↓
CSV reconciled data
↓
PostgreSQL reconciled database
↓
Data Quality Assessment
↓
Data cleaning
↓
ETL toward the Data Warehouse
↓
Star schema and OLAP analysis

General Data Quality Assessment

It is important to distinguish three different activities: Data Quality Assessment, Data Cleaning, and Analytical Filtering. They are related, but they have different goals.

Activity	**Goal**	**Example in this project**
Data Quality Assessment	Measure the current quality level and identify problems.	Count missing values in `lap`, detect duplicate keys, find invalid sector times.

Activity	Goal	Example in this project
Data Cleaning	Correct, remove, or standardize data quality problems.	Remove deleted or inaccurate laps, standardize textual values.
Analytical Filtering	Select the subset of valid data useful for the final story.	Keep only Ferrari, Red Bull, and Mercedes; keep only dry conditions.

Theoretical Background

Data quality is a multidimensional concept. For this reason, a single global value is not sufficient to understand whether a dataset is reliable.

The assessment in this project follows the ISO 25012 perspective introduced in the course material. The six main dimensions considered are:

- Completeness
- Uniqueness
- Validity
- Consistency
- Accuracy
- Timeliness

Since this project is based on a relational PostgreSQL reconciled database, an additional practical dimension is also considered:

- Referential Integrity

This additional dimension is essential before adding SQL foreign key constraints and before designing the ETL toward the Data Warehouse.

Completeness Completeness measures whether required values are present. In practical terms, a completeness check verifies the percentage of non-null values for attributes that are necessary to identify, connect, or analyze the records.

In this project, completeness is applied differently depending on the role of each attribute. Structural identifiers such as `lap_id`, `session_id`, `driver_id`, and `team_id` are expected to be complete. Analytical attributes such as `lap_time_ms` may be missing in some special cases and must be evaluated more carefully.

Formula used conceptually:

$$Completeness(A) = \frac{\#\,non\text{-}null\ values\ in\ A}{\#\,total\ rows}$$

Uniqueness Uniqueness measures whether keys and candidate keys are free from duplicates. This dimension is crucial because the reconciled database must later support primary key constraints and reliable joins.

In this project, uniqueness is checked both on surrogate identifiers and on natural keys. For example, `lap_id` must be unique if it is used as surrogate key, while the combination `session_id + driver_id + lap_number` should also identify a lap naturally.

Formula used conceptually:

$$Uniqueness(K) = \frac{\#\,distinct\ key\ values}{\#\,total\ key\ values}$$

Validity Validity measures whether values conform to allowed domains, formats, and business rules. A value may be present and unique, but still invalid if it violates the expected domain.

Consistency Consistency measures whether values are coherent with each other. This type of check usually compares two or more attributes in the same table, or values stored in different related tables.

Accuracy and Plausibility Accuracy measures whether values reflect the real-world truth. This dimension is difficult to verify automatically without an external reference source. For this reason, in this project accuracy is interpreted mainly as plausibility.

A plausibility check does not always prove that a value is wrong. Instead, it identifies suspicious values that require further inspection.

Timeliness Timeliness measures whether data are temporally suitable for their intended use. Since this project analyzes historical Formula 1 seasons, timeliness does not mean that the data must be updated to the current day. Instead, it means that dates must be coherent with the temporal scope of the analysis.

Referential Integrity Referential Integrity verifies that every foreign key value in a child table points to an existing row in the parent table.

Assessment Methodology

The Data Quality Assessment is implemented through a Python script connected to the PostgreSQL reconciled database. The script reads the tables, applies deterministic checks, and exports both aggregated scores and detailed issue files.

DA AGGIUNGERE IN FUTURO =¿ COSA è OUTPUT E COSA SIGNIFICA

Score Interpretation

The scorecard uses a simple interpretation scale:

Score Range	Status	Meaning
$score \geq 0.95$	Green	The quality level is acceptable. Only minor monitoring is required.
$0.80 \leq score < 0.95$	Yellow	The quality level is acceptable for exploration, but cleaning should be planned.
$score < 0.80$	Red	The quality problem is critical and may block downstream usage.

Main Output Files

The assessment script is expected to produce the following outputs:

Output file	Purpose
dq_scorecard_all_tables.csv	Detailed scorecard with one row for each table, dimension, and check.
dq_scorecard_by_table.csv	Summary score by table, useful to identify the weakest tables.
dq_table_profiles.csv	Basic profiling information such as number of rows, number of columns, and missing values.
dq_issues_lap.csv	Row-level issues detected in the lap table.
dq_issues_result.csv	Row-level issues detected in the result table.
dq_issues_referential_integrity.csv	Broken references between child and parent tables.
dq_scorecard_for_llm.json	Compact JSON version of the scorecard, useful only for generating a natural-language explanation.

Table-by-Table Data Quality Plan

This section defines the planned checks for each table of the reconciled database. The actual numerical results will be inserted after running the Python assessment script.

Season Table

Although the season table was manually created from domain knowledge and contains only two records, it is still included in the Data Quality Assessment. The goal is to verify that this manually introduced domain table is complete, unique, and semantically coherent with the Formula 1 domain.

The restriction to the 2021–2022 seasons is not considered a validity rule. Other Formula 1 seasons are valid domain values, but they are outside the analytical scope of this project.

Dimension	Checks
Completeness	season_year must be present because it is the structural identifier of the season.
Uniqueness	season_year must be unique, since it is the primary key of the table.
Validity	season_year must belong to the Formula 1 historical domain, starting from 1950. If present, number_of_events must be greater than zero.
Consistency	If both dates are present, season_start_date must be before season_end_date. Moreover, the year component of the season dates must be coherent with season_year.
Accuracy/Plausibility	Full accuracy of championship-level attributes, such as champion driver or champion team, is not implemented as an automatic check in the DQA script. Since the table is manually introduced, its accuracy is documented through source provenance during the re-engineering phase.
Timeliness	Not applicable as a freshness check, because the table represents closed historical seasons. Temporal coherence is already evaluated through consistency checks on the date attributes.

Circuit Table

The `circuit` table is partly based on domain knowledge and includes derived categorical attributes such as the global circuit category and the sector categories.

This table is used to support the analytical interpretation of performance across different circuit types. For this reason, the Data Quality Assessment focuses on the completeness and validity of the circuit classification attributes.

Dimension	Checks
Completeness	`circuit_id`, `global_circuit_category`, `sector1_category`, `sector2_category`, and `sector3_category` must be present. These attributes are required because they identify the circuit and provide the analytical classification used in the project. Other descriptive attributes, such as `circuit_name`, `country`, `location`, and `seasons_present`, are not treated as required columns in the general DQA because they are not essential for the analytical classification.
Uniqueness	`circuit_id` must be unique, since it is the primary key of the table.
Validity	`global_circuit_category` must belong to the allowed global circuit category domain. `sector1_category`, `sector2_category`, and `sector3_category` must belong to the allowed sector category domain. These checks are applied only when the values are present, because missing required values are already handled by the completeness checks.
Consistency	No automatic consistency checks are implemented for this table. Textual standardization, such as trimming and empty-string handling, is managed by general cleaning rules. Geographical consistency between `country` and `location` is not checked automatically because it would require an external geographical reference source and is outside the scope of the general DQA script.
Accuracy/Plausibility	Not implemented as a fully automatic check. The plausibility of the circuit and sector classifications is documented through the manual classification process and source provenance used during the re-engineering phase.
Timeliness	Not applicable as a freshness check. Possible future circuit layout changes are outside the scope of this assessment, which is limited to the selected historical seasons.

Allowed domains:

- Global circuit category: `Street`, `PowerSensitive`, `AeroSensitive`, `Mixed`
- Sector category: `Power`, `FastCorners`, `SlowCorners`, `Technical`

Driver Table

The `driver` table contains stable driver master data obtained from the Ergast/Jolpica source through the FastF1 interface. It is used as a reference table to identify drivers in result-level and lap-level data.

Since this table is a master table, the Data Quality Assessment focuses mainly on structural completeness, key uniqueness, and a small set of domain checks on descriptive attributes.

Dimension	Checks
Completeness	`driver_id`, `abbreviation`, and `last_name` must be present. `driver_id` is required to identify the driver and to support joins with results and laps. `abbreviation` and `last_name` are required because they provide readable labels for reports and visual analyses. Other descriptive attributes, such as `first_name`, `date_of_birth`, `nationality`, `permanent_number`, and `driver_url`, are not treated as required structural columns in the general DQA.
Uniqueness	`driver_id` must be unique, since it is the primary key of the table.
Validity	`abbreviation` must follow the expected driver-code format, namely a three-letter uppercase alphabetic code. If present, `permanent_number` must be between 1 and 99. If present, `driver_url` must have a valid URL format. Date parsing for `date_of_birth` is not checked here, because type validation is handled during the typed PostgreSQL loading phase.
Consistency	No specific consistency checks are implemented for this table. Textual standardization, such as trimming and empty-string handling, is managed by general cleaning rules applied to textual attributes.
Accuracy/Plausibility	Not implemented as a fully automatic truth check. If `date_of_birth` is present, it can be used to flag implausible driver ages during the selected 2021–2022 seasons. This is treated as a broad plausibility check, not as strict validity, because unusual ages would require domain interpretation rather than automatic correction.
Timeliness	Not applicable as a freshness check, because the table contains historical driver reference data for the selected seasons.

Team Table

The `team` table contains constructor/team master data obtained from the Ergast/Jolpica source through the FastF1 interface. It is used as a reference table to identify teams in result-level and lap-level data.

Basic textual standardization, such as trimming spaces and converting empty strings into null values, is handled by general cleaning rules and is not repeated as a specific validity check for this table.

Dimension	Checks
Completeness	`team_id` and `team_name` must be present. `team_id` is required because it identifies the team in joins, while `team_name` is required as the readable business label used in reports and visual analyses.
Uniqueness	`team_id` must be unique, since it is the primary key of the table.
Validity	If present, `team_url` must have a valid URL format. The format of `team_id` is not checked as a separate validity rule, because it is inherited from the source identifier and is already controlled through completeness and uniqueness checks.

Dimension	Checks
Consistency	No specific consistency checks are implemented for this table. Basic textual normalization, such as trimming and empty-string handling, is managed through general cleaning rules applied to all textual attributes.
Accuracy/Plausibility	Not implemented as an automatic truth check. Team names and nationalities are documented through source provenance from Ergast/Jolpica during the re-engineering phase.
Timeliness	Not applicable as a freshness check, because the table contains historical reference data for the selected seasons.

Grand Prix Table

The `grand_prix` table represents a Formula 1 Grand Prix event in a specific season. Each Grand Prix belongs to one season and is associated with one circuit.

This table is derived from the re-engineered FastF1 event schedule. During the re-engineering phase, each event was enriched with `circuit_id` in order to connect the Grand Prix to the corresponding circuit.

Dimension	Checks
Completeness	`grand_prix_id`, `season_year`, `round_number`, `event_name`, `event_format`, and `circuit_id` must be present. These attributes are required to identify the Grand Prix, connect it to the corresponding season, preserve its order within the season, provide a readable event label, distinguish the event format, and associate the event with the circuit where it takes place.
Uniqueness	`grand_prix_id` must be unique, since it is the surrogate primary key of the table. The natural key `season_year + round_number` should also be unique, because a round number identifies one Grand Prix within a specific season.
Validity	`round_number` must be greater than zero. `event_format` must belong to the allowed Grand Prix event format domain, such as `conventional` or `sprint`.
Consistency	If `event_date` is present, it must be coherent with `season_year`.
Accuracy/Plausibility	Not implemented. The association between each Grand Prix and its circuit is produced during the re-engineering phase through a deterministic matching rule based on `country` and `location`. For this reason, the DQA does not repeat the matching logic. It only verifies that the resulting `circuit_id` is present and referentially valid.
Timeliness	Not applicable as a freshness check.
Referential Integrity	`season_year` must exist in the `season` table and `circuit_id` must exist in the `circuit` table.

Session Table

The `session` table represents a Qualifying or Race session belonging to a specific Grand Prix. Free practice sessions are outside the analytical scope of the project and are excluded during the

re-engineering phase.

Each session is connected to one Grand Prix through `grand_prix_id`. The attribute `session_type` identifies the type of session considered in the project, namely Qualifying or Race.

Dimension	**Checks**
Completeness	`session_id`, `grand_prix_id`, and `session_type` must be present. These attributes are required to identify the session, connect it to the corresponding Grand Prix, and define the type of session analyzed in the project.
Uniqueness	`session_id` must be unique, since it is the surrogate primary key of the table. The natural key `grand_prix_id + session_type` should also be unique, because the same Grand Prix should contain at most one session of each selected type in the reconciled schema.
Validity	`session_type` must belong to the allowed session domain for this project, namely `Q` for Qualifying or `R` for Race.
Consistency	If `session_date` is present, it must be temporally coherent with the corresponding Grand Prix `event_date`. Since `event_date` represents the main date of the Grand Prix event, a Qualifying or Race session should normally occur on the same day or shortly before it. In practice, the check allows `session_date` to fall between `event_date` minus two days and `event_date`. This also covers sprint weekends, where Qualifying may occur earlier than in a conventional weekend.
Accuracy/Plausibility	Not implemented as an automatic truth check.
Timeliness	Not applicable as a freshness check.
Referential Integrity	`grand_prix_id` must exist in the `grand_prix` table. This check is important before adding explicit SQL foreign key constraints to the reconciled database.

Result Table

The `result` table represents the result obtained by one driver in one specific session. It is structurally important because it connects drivers and teams to sessions and also supports the enrichment of lap-level data.

This table contains both qualifying-related attributes, such as `q1_ms`, `q2_ms`, and `q3_ms`, and race-related attributes, such as `classified_position`, `status`, `points`, and `laps`. For this reason, completeness checks must distinguish between structural columns and session-dependent analytical columns.

Dimension	Checks
Completeness	`result_id`, `session_id`, `driver_id`, `team_id`, and `position` must be present because they identify the result, connect it to the corresponding session, driver, and team, and provide the final ranking position within the session. For race sessions, `classified_position`, `status`, `points`, and `laps` are also checked as required race-result context attributes. By contrast, `q1_ms`, `q2_ms`, and `q3_ms` are not treated as generally required attributes, because their null values may be expected depending on qualifying progression or non-participation.
Uniqueness	`result_id` must be unique, since it is the surrogate primary key of the table. The natural key `session_id + driver_id` should also be unique, because the same driver should have at most one result in a specific session.
Validity	`position` must be greater than zero when present. `points` and `laps` must be greater than or equal to zero when present. If present, `grid_position` must also be greater than or equal to zero.
Consistency	Qualifying time attributes must be interpreted according to the qualifying progression. For example, if `q3_ms` is present, then the driver is expected to have also reached the previous qualifying phases; similarly, if `q2_ms` is present, `q1_ms` should normally be present.
Accuracy/Plausibility	Not implemented as an automatic truth check.
Timeliness	Not applicable as a freshness check.
Referential Integrity	`session_id`, `driver_id`, and `team_id` must reference existing rows in `session`, `driver`, and `team`. These checks are important before adding explicit SQL foreign key constraints to the reconciled database.

Lap Table

Each row of the `lap` table represents one lap performed by one driver in one specific session.

For this reason, the Data Quality Assessment distinguishes between structural completeness and analytical missing values.

Not all missing analytical values are automatically considered data quality errors. For example, `lap_time_ms` or sector times may be missing when a lap is not completed, when the driver enters or exits the pit lane, when the lap is deleted or inaccurate, or when special racing conditions occur. Therefore, detailed missing value interpretation for performance attributes is handled in the focused missing values analysis.

Dimension	Checks
Completeness	`lap_id`, `session_id`, `driver_id`, `team_id`, `lap_number`, and `time_ms` must be present. These attributes identify the lap, connect it to the corresponding session, driver, and team, and place it on the session timeline.

Dimension	Checks
Uniqueness	`lap_id` must be unique, since it is the surrogate primary key of the table. The natural key `session_id + driver_id + lap_number` should also be unique, because the same driver cannot have two different laps with the same lap number inside the same session.
Validity	`lap_number` must be greater than zero. If present, `lap_time_ms` and sector times must be positive values. Speed attributes such as `speed_i1`, `speed_i2`, `speed_fl`, and `speed_st` must be positive when present. If present, `stint` and `tyre_life` must be positive values. `compound` must belong to the expected tyre compound domain.
Consistency	For timed laps with complete sector information, the sum of `sector1_time_ms`, `sector2_time_ms`, and `sector3_time_ms` should be coherent with `lap_time_ms`, allowing a small tolerance for rounding or source precision. This rule appear only when the all three sectors are present.
Accuracy/Plausibility	Broad performance plausibility is not implemented as a general DQA rule for this table, because suspicious lap times, sector times, or speed values may depend on pit stops, Safety Car periods, Virtual Safety Car periods, red flags, rain, deleted laps, inaccurate laps, or other special racing situations. For this reason, suspicious numerical values are analyzed separately in the focused outlier detection phase, where they are flagged for review rather than automatically considered wrong.
Timeliness	Temporal quality is evaluated through the presence of `time_ms` and the coherence of relative time attributes within the session timeline.
Referential Integrity	`session_id`, `driver_id`, and `team_id` must reference existing rows in the `session`, `driver`, and `team` tables.

Weather Table

Each row of the `weather` table represents one weather observation associated with a session and identified by a relative session time.

Dimension	Checks
Completeness	`weather_id`, `session_id`, `time_ms`, `air_temp`, `track_temp`, `rainfall`, and `wind_speed` must be present. The first three attributes are required to identify each weather observation, connect it to the corresponding session, and place it on the session timeline. The remaining attributes are the main weather measures used to describe the environmental context of the session.
Uniqueness	`weather_id` must be unique, since it is the surrogate primary key of the table. The natural key `session_id + time_ms` should also be unique, because a session should not contain two different weather observations with the same timestamp.

Dimension	Checks
Validity	`time_ms` must be greater than or equal to zero. Numerical weather attributes must belong to broad physical domain ranges. If present, `air_temp` and `track_temp` must be within broad Formula 1 weather ranges, `humidity` must be between 0 and 100, `pressure` must be within a broad realistic atmospheric range, `wind_direction` must be between 0 and 360 degrees, and `wind_speed` must be greater than or equal to zero.
Consistency	`time_ms` represents elapsed time from the beginning of the session and must therefore be coherent with the session timeline. Weather variables are also checked for internal coherence. In particular, `track_temp` should be reasonably coherent with `air_temp` at maximum 50 degree. A large difference between the two values is flagged as suspicious, but not automatically treated as an error, because track temperature can be significantly higher than air temperature in hot conditions.
Accuracy/Plausibility	Not implemented as a truth check against an external meteorological source.
Timeliness	Not applicable as a freshness check.
Referential Integrity	`session_id` must reference an existing row in the `session` table.

Track Status Table

The `track_status` table contains time-varying track condition messages recorded during a specific Formula 1 session. Each row represents a change or state of the track at a given elapsed time within the session.

Like the `weather` table, `track_status` is not a direct to-one branch of the Lap Performance attribute tree, because a session can have many track status records over time. However, it is analytically useful because it can be used during ETL to derive lap-level or session-level context attributes.

Dimension	Checks
Completeness	`track_status_id`, `session_id`, `time_ms`, `status`, and `message` must be present. The first three attributes are required to identify the record, connect it to a session, and place it on the session timeline. The attributes `status` and `message` are also required because they contain the actual track condition information. Without them, the row would not provide useful analytical context.
Uniqueness	`track_status_id` must be unique, since it is the surrogate primary key of the table. The natural key `session_id + time_ms` should also be checked, because a session should not contain multiple track status records with the same timestamp unless this is explicitly justified by the source.
Validity	`time_ms` must be greater than or equal to zero. `status` must belong to the documented set of track status codes, and `message` must belong to the documented set of track status messages.

Dimension	Checks
Consistency	`time_ms` represents elapsed time from the beginning of the session, therefore it must progress coherently within each `session_id`. In addition, `status` and `message` must be coherent with each other: the same numerical status code should always correspond to the same textual message. For example, a status code representing clear track conditions should not be associated with a Safety Car or Red Flag message.
Accuracy/Plausibility	Not implemented as a truth check against an external race-control source. Suspicious cases, such as unknown status codes, unknown messages, or inconsistent status-message pairs, are flagged for review.
Timeliness	Not applicable as a freshness check.
Referential Integrity	`session_id` must reference an existing row in the `session` table.

Expected status-message domain:

Status code	Message	Meaning
1	AllClear	Normal track condition.
2	Yellow	Yellow flag condition.
4	SCDeployed	Safety Car deployed.
5	Red	Red flag condition.
6	VSCDeployed	Virtual Safety Car deployed.
7	VSCEnding	Virtual Safety Car ending phase.

Planned Visual Outputs

The Python assessment script should also generate visual outputs to summarize the results. These figures will be inserted in the final version of the report after running the script.

Overall Data Quality Score by Table

This plot compares the average data quality score of each table. It is useful to identify which table requires the most cleaning effort.

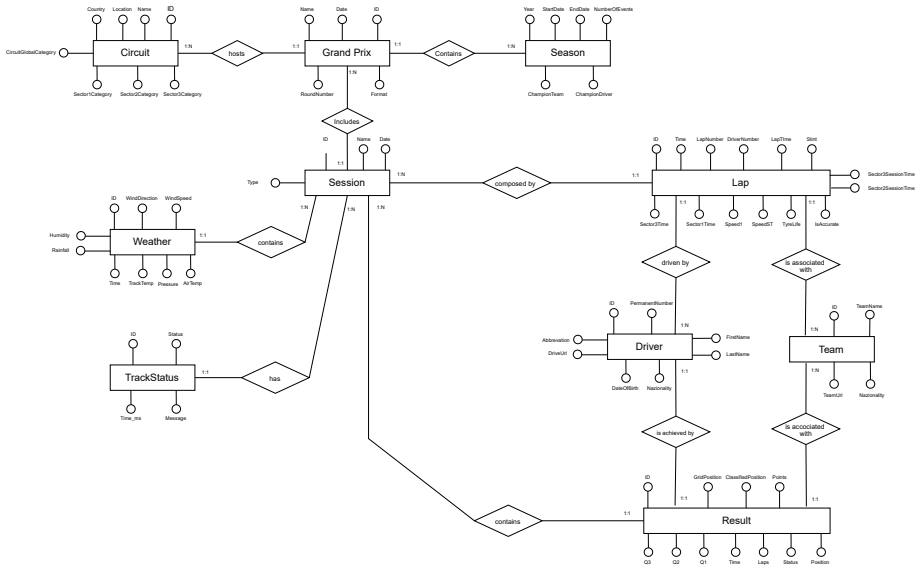

Figure 1: Overall Data Quality Score by Table

Data Quality Score by Dimension

This plot compares the quality dimensions across the whole database. It is useful to understand whether the main problem is completeness, uniqueness, validity, consistency, timeliness, accuracy, or referential integrity.

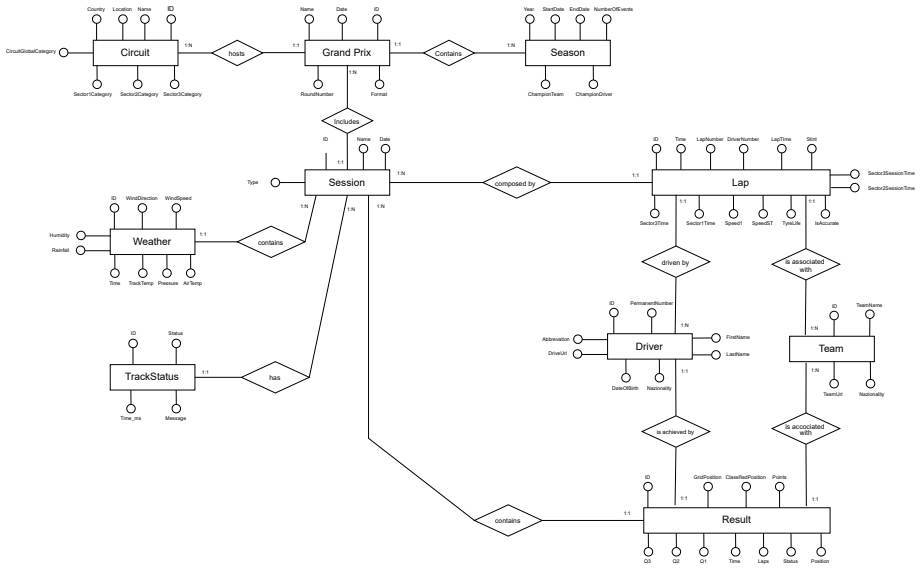

Figure 2: Data Quality Score by Dimension

Heatmap of Table-Dimension Scores

A heatmap is useful to show which tables and dimensions have the weakest scores. This figure allows the reader to identify problematic combinations, such as low completeness in `lap` or low validity in `weather`.

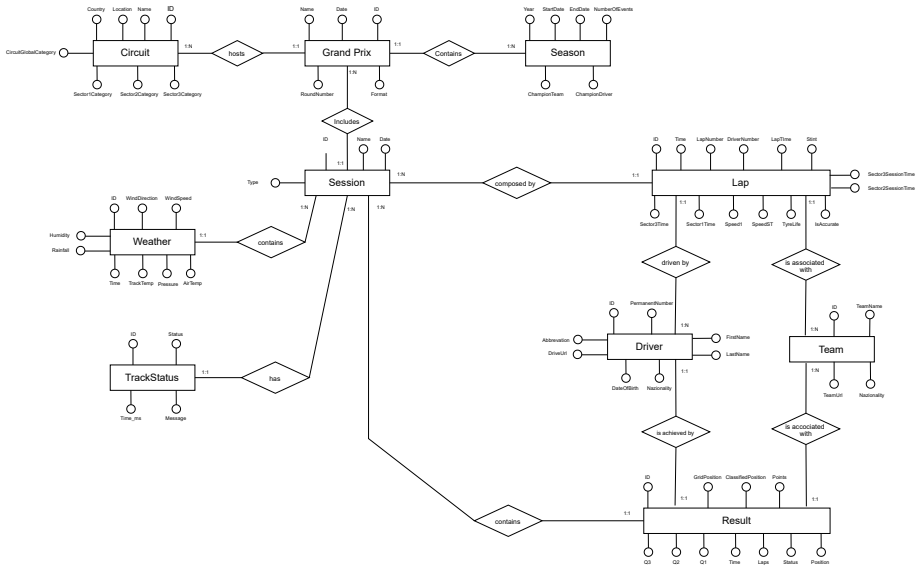

Figure 3: Heatmap of Data Quality Scores by Table and Dimension

Missing Values Summary

This plot summarizes missing values by table or by column. It supports the completeness analysis and helps identify attributes that require special attention during cleaning.

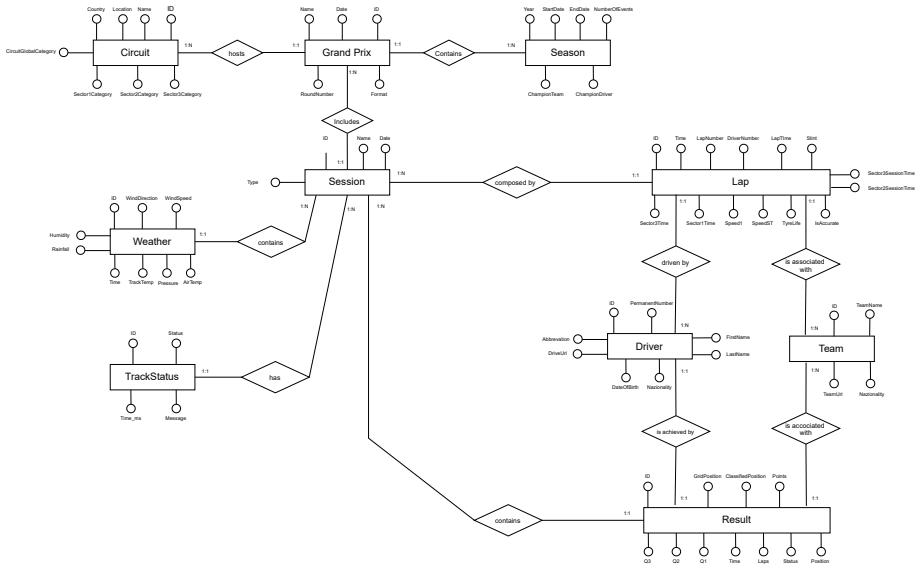

Figure 4: Missing Values Summary

Referential Integrity Issues

This plot summarizes broken references between child and parent tables. It is especially important before applying explicit SQL foreign key constraints.

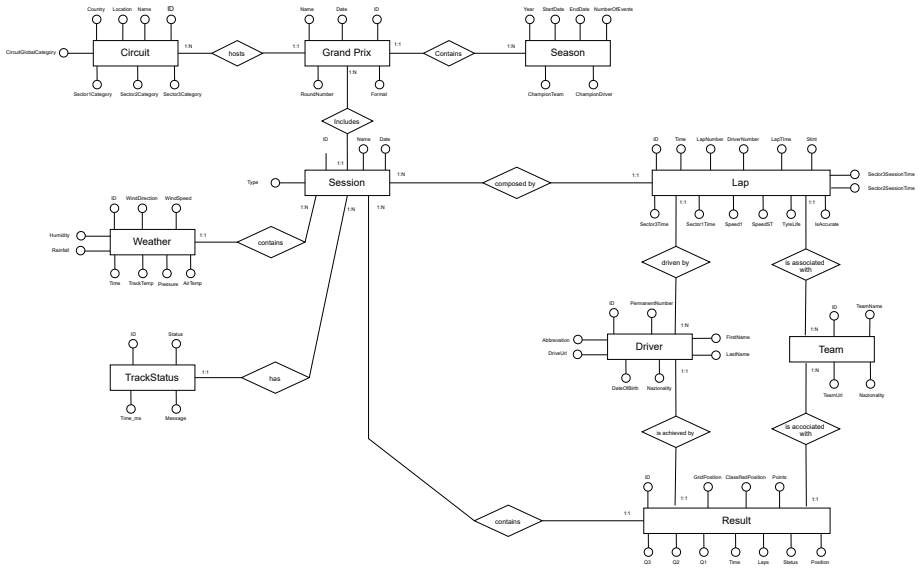

Figure 5: Referential Integrity Issues by Relationship

Focused Missing Values Analysis

Missing values are treated as a focused data quality problem mainly related to the *Completeness* dimension. However, in this project, not every null value is automatically considered a data quality error. Some attributes may be null because the information is not applicable in a specific Formula 1 context.

For this reason, the missing values analysis does not simply count null values. It also classifies them according to their meaning in the domain.

Missing Values and Completeness

The first interpretation of missing values is connected to completeness. A table is complete when all required values are present. In this project, structural identifiers and foreign keys are considered mandatory, because they are necessary to connect the reconciled tables.

Examples of structural attributes are:

- `session_id`
- `driver_id`
- `team_id`
- `lap_id`
- `result_id`

If one of these attributes is missing, the row may become unusable for relational joins and for the later Data Warehouse loading phase.

By contrast, some analytical attributes may be missing without immediately invalidating the row. For example, a missing `lap_time_ms` must be investigated, but it is not equivalent to a missing primary or foreign key.

MCAR, MAR, and MNAR Classification

The theoretical classification of missing values distinguishes between three main mechanisms: MCAR, MAR, and MNAR.

Mechanism	Meaning	Interpretation in this project
MCAR	Missing Completely At Random. The missingness has no relationship with other variables in the dataset.	Possible case for small random gaps in contextual data, such as occasional missing weather observations.
MAR	Missing At Random. The missingness depends on other observed variables.	Possible case for qualifying times, where `q2_ms` or `q3_ms` may be missing depending on the driver's progression through the qualifying session.
MNAR	Missing Not At Random. The missingness depends on the missing value itself or on an unobserved reason.	Possible suspicious case when missing lap times are related to problematic laps, deleted laps, inaccurate laps, or special racing conditions.

This classification is useful because different missing mechanisms require different cleaning strategies. In this project, missing values are not automatically imputed. Instead, they are first classified and then used to define motivated cleaning or filtering decisions.

Expected NULL Values

Some null values are expected and should not be interpreted as data quality errors. These values are null because the corresponding attribute is meaningful only under specific conditions.

Examples include:

Attribute	Condition	Interpretation
`q1_ms`, `q2_ms`, `q3_ms`	Session type is Race	Expected null, because qualifying segment times are not applicable to race sessions.
`q2_ms`	Driver did not reach Q2	Expected null, because the driver has no Q2 time.
`q3_ms`	Driver did not reach Q3	Expected null, because the driver has no Q3 time.
`pit_in_time_ms`	Normal lap	Expected null, because the lap is not an in-lap.
`pit_out_time_ms`	Normal lap	Expected null, because the lap is not an out-lap.
`deleted_reason`	Lap is not deleted	Expected null, because no deletion reason exists.

Therefore, the missing values analysis must distinguish between expected nulls and real data quality problems.

Problematic NULL Values

A null value is considered problematic when it affects the identification, connection, or analytical usability of a row.

Examples of problematic null values are:

- missing primary keys or surrogate identifiers;
- missing foreign keys required to join tables;
- missing `driver_id` or `team_id` in fact-like tables;
- missing `session_id` in session-dependent tables;
- missing `lap_number` in the lap table;
- missing `lap_time_ms` in laps that should represent valid timed laps.

These missing values are flagged by the assessment script and are later considered during the cleaning phase.

The output of this analysis is expected to include:

- a summary of missing values by table;
- a summary of missing values by column;
- a classification of expected and problematic null values;
- a list of rows requiring further inspection.

Focused Outlier Detection

Outlier detection is treated as a focused data quality analysis related mainly to *Accuracy*, *Validity*, and *Consistency*. The goal is to identify values that are statistically unusual or suspicious from a domain perspective.

In this project, an outlier is not automatically considered a wrong value. Formula 1 data can contain extreme but valid values due to pit stops, Safety Car periods, red flags, weather conditions, deleted laps, inaccurate laps, or other special racing situations.

Therefore, outlier detection is used to flag suspicious records, not to automatically delete them.

IQR Method

The Interquartile Range method is a simple and interpretable univariate outlier detection technique. It is based on the first quartile, the third quartile, and the interquartile range.

$$IQR = Q3 - Q1$$

A value is considered suspicious if it falls outside the following interval:

$$LowerBound = Q1 - k \cdot IQR$$

$$UpperBound = Q3 + k \cdot IQR$$

where k is usually set to 1.5 for standard outlier detection or to a larger value for a more conservative analysis.

In this project, the IQR method can be applied to numerical attributes such as:

- `lap_time_ms`

- `sector1_time_ms`
- `sector2_time_ms`
- `sector3_time_ms`
- speed attributes
- weather attributes

Modified Z-score

The Modified Z-score is a robust alternative to the classical Z-score. It uses the median and the Median Absolute Deviation instead of the mean and standard deviation.

$$M_i = \frac{0.6745(x_i - median(x))}{MAD}$$

where:

$$MAD = median(|x_i - median(x)|)$$

A value is usually considered suspicious when:

$$|M_i| > 3.5$$

This method is useful because Formula 1 performance data may be skewed. For example, lap times may include normal fast laps, slow laps, pit laps, laps under Safety Car, and other special cases. Using the median makes the method less sensitive to extreme values.

Consensus and Multi-method Flag

No single outlier detection method is perfect. For this reason, the assessment uses a simple consensus strategy.

Each numerical value can be flagged by one or more methods. The number of methods that flag the same value is used as an indicator of anomaly strength.

Number of Flags	**Interpretation**	**Action**
0	Normal value	No action required.
1	Weak anomaly	Review only if the attribute is important for the analysis.
2 or more	Stronger suspicious value	Investigate before using the row in the Data Warehouse.

In the first implementation, the consensus is based on the IQR method and the Modified Z-score. More advanced techniques, such as Isolation Forest or Local Outlier Factor, can be added later if necessary.

Domain Interpretation

The statistical detection of an outlier is not sufficient to decide whether a record must be removed or corrected. Each suspicious value must be interpreted in the Formula 1 domain.

For example, a very slow lap can be caused by:

- an in-lap;

- an out-lap;
- a pit stop;
- a Safety Car period;
- a Virtual Safety Car period;
- rain or wet track conditions;
- red flags or yellow flags;
- an inaccurate or deleted lap.

For this reason, the output of the outlier detection script is used as input for later cleaning and analytical filtering decisions. The script does not delete outliers automatically. It only flags them and provides evidence for manual or rule-based interpretation.

The expected outputs are:

- an outlier summary by table and column;
- row-level outlier flags;
- a consensus score for suspicious values;
- plots showing outlier distributions;
- a list of records requiring further review.

Optional LLM Support

A Large Language Model can be used as an optional support layer for interpreting the Data Quality Assessment outputs. However, it is not part of the deterministic data quality pipeline and it does not replace the Python-based checks.

The official assessment results are produced only by deterministic scripts. These scripts compute quality scores, detect missing values, identify outliers, and generate row-level issue files. The LLM can only be used after these outputs have been generated.

Not Part of the Deterministic Pipeline

The LLM is not used to decide whether a value is valid, whether a row must be removed, or whether a cleaning rule must be applied.

The deterministic part of the pipeline includes:

- general data quality checks;
- missing values analysis;
- outlier detection;
- referential integrity checks;
- row-level flags;
- scorecard generation.

These steps are reproducible because the same input data and the same rules always produce the same outputs.

By contrast, LLM outputs are probabilistic and may vary depending on the prompt, model, and configuration. For this reason, the LLM is separated from the official assessment logic.

Used Only for Explanation

If used, the LLM receives only aggregated and already generated outputs, such as:

- data quality scorecards;
- missing value summaries;
- missing value classifications;
- outlier summaries;
- referential integrity issue summaries.

The goal is to generate a natural-language explanation of the results. Possible outputs include:

- a stakeholder-oriented data quality summary;
- a technical explanation of the main quality problems;
- a draft paragraph for the final report;
- a preliminary list of cleaning priorities.

The LLM does not access the PostgreSQL database directly and does not modify any data.

Manually Validated Output

Every LLM-generated output must be manually reviewed before being included in the project documentation.

The LLM output is treated as a draft, not as an authoritative result. The final explanation included in the report must be checked against the original scorecards and issue files.

For transparency, the following files can be stored:

- `llm_input_dqa_summary.json`
- `llm_prompt_dqa_interpretation.txt`
- `llm_output_dqa_interpretation.md`
- `llm_human_review_notes.md`

This makes the use of the LLM explicit, controlled, and auditable.

In summary, the LLM is used only as an explanatory assistant. The official data quality decisions remain based on deterministic and reproducible checks.

Conclusion

The final goal is not only to detect problems, but also to produce evidence for the next phase. The generated scorecards, issue files, and plots will guide the cleaning script and will support the documentation of all data quality decisions in the final project report.